Digital System Design - PA12

Image Convolution using VHDL



Deskripsi Proyek

Latar Belakang

- Image processing adalah sebuah operasi yang sangat intensif dalam hal komputasi-nya.
- Hardware acceleration dengan FPGA memberikan efisiensi yang lebih tinggi dibandingkan perangkat lunak beserta CPU biasa.

Tujuan Proyek

- Mengembangkan sistem *image convolution* berbasis FPGA.
- Mengintegrasikan ALU (Arithmetic Logic Unit) dan MMU (Matrix Multiplier Unit) untuk mendukung berbagai algoritma dalam melakukan *convolution*.
- Mendukung pemilihan kernel secara dinamis melalui instruksi yang berbeda.



Dasar Teori

Convolution

- Operasi matematika untuk memfilter atau memodifikasi gambar. Operasi ini berguna untuk menggabungkan hasil dari dua fungsi.
- Rumus dasar *convolution* sebagai berikut.

$$F(x,y) = \sum_i \sum_j f(x+i,\ y+j)\ h(i,j)$$

Kernel

- Matriks 3x3, 5x5, atau 7x7 berfungsi untuk memodifikasi pixel sehingga menghasilkan *Feature Map*.
- Contoh: Box Blur, Sobel, Sharpening, Laplacian, dll.



Dasar Teori

Jenis Algoritma Convolution

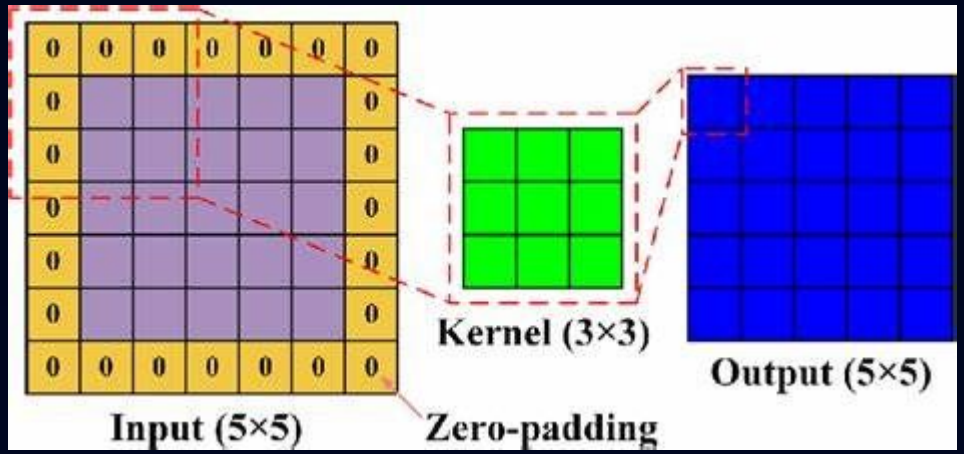
- **Direct Convolution** : Metode dasar untuk melakukan operasi *convolution*, di mana setiap elemen dalam filter (kernel) digeser melintasi elemen input (pixel pada gambar) dan dihitung satu per satu.
- **Im2col Convolution** : Metode ini mengubah input menjadi representasi matriks (disebut im2col, dari *image to column*) sebelum operasi *convolution* dilakukan. Lalu, menggunakan operasi matriks untuk menghitung hasil *convolution* secara bersamaan.



Dasar Teori

Zero Padding

Teknik dalam image processing di mana nilai nol ditambahkan di sekitar tepi data, seperti gambar. Teknik ini dilakukan untuk mengontrol dimensi output.





Dasar Teori

Direct Convolution

26	0	26	26	26	26
0	26	0	26	26	26
0	26	0	26	26	26

0	0	0	0	0	0	0	0
0	26	0	26	26	26	26	0
0	0	26	0	26	26	26	0
0	0	26	0	26	26	26	0
0	0	0	0	0	0	0	0

52	78	104	130	156	104
78	104	156	182	234	156
52	52	104	104	156	104

26	26	26	26	0	26
26	26	26	26	0	26
26	26	26	0	0	0

0	0	0	0	0	0	0	0
0	26	26	26	26	0	26	0
0	26	26	26	26	0	26	0
0	26	26	26	0	0	0	0
0	0	0	0	0	0	0	0

104	156	156	104	104	52
156	234	208	130	104	52
104	156	130	78	52	26

0	26	0	0	26	0
26	0	26	0	26	0
26	0	26	26	26	26

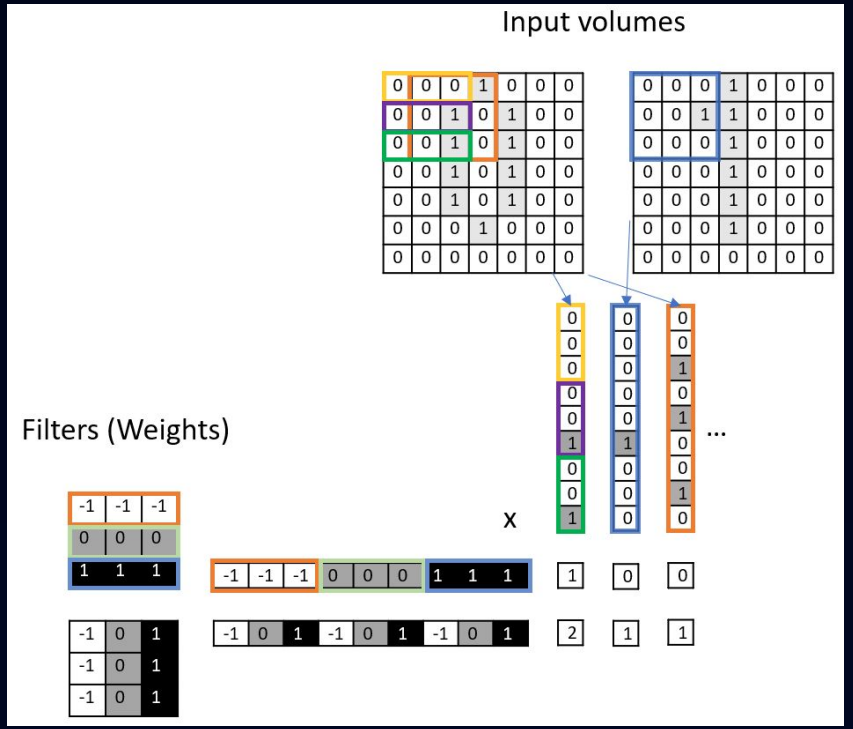
0	0	0	0	0	0	0	0
0	0	26	0	0	26	0	0
0	26	0	26	0	26	0	0
0	26	0	26	26	26	26	0
0	0	0	0	0	0	0	0

52	78	52	78	52	52
78	130	104	156	130	104
52	104	78	130	104	78

1	1	1
1	1	1
1	1	1

Dasar Teori

Im2col Convolution



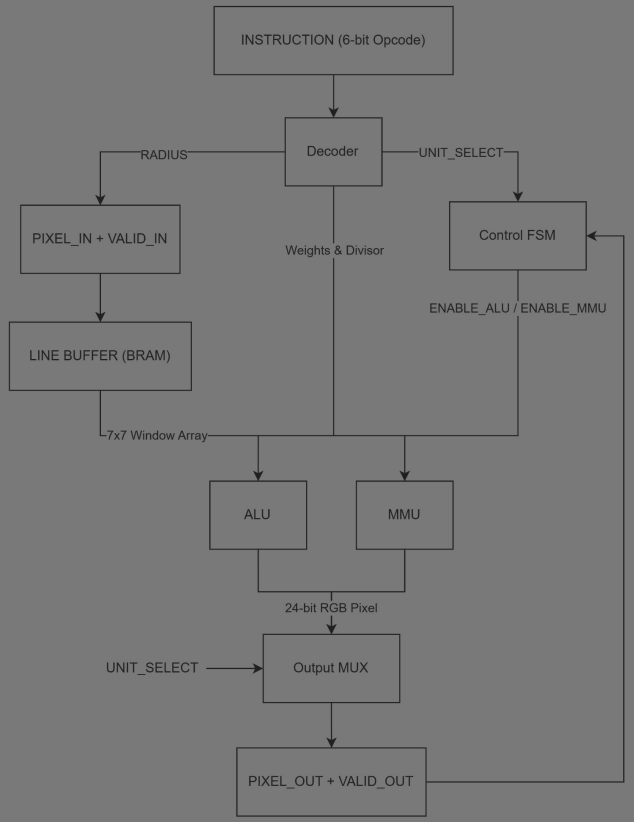
Offset	Size	Hex value	Value	Description
BMP Header				
0h	2	42 4D	"BM"	ID field (42h, 4Dh)
2h	4	46 00 00 00	70 bytes (54+16)	Size of the BMP file (54 bytes header + 16 bytes data)
6h	2	00 00	Unused	Application specific
8h	2	00 00	Unused	Application specific
Ah	4	36 00 00 00	54 bytes (14+40)	Offset where the pixel array (bitmap data) can be found
DIB Header				
Eh	4	28 00 00 00	40 bytes	Number of bytes in the DIB header (from this point)
12h	4	02 00 00 00	2 pixels (left to right order)	Width of the bitmap in pixels
16h	4	02 00 00 00	2 pixels (bottom to top order)	Height of the bitmap in pixels. Positive for bottom to top pixel order.
1Ah	2	01 00	1 plane	Number of color planes being used
1Ch	2	18 00	24 bits	Number of bits per pixel
1Eh	4	00 00 00 00	0	BI_RGB, no pixel array compression used
22h	4	10 00 00 00	16 bytes	Size of the raw bitmap data (including padding)
26h	4	13 0B 00 00	2835 pixels/metre horizontal	Print resolution of the image, 72 DPI × 39.3701 inches per metre yields 2834.6472
2Ah	4	13 0B 00 00	2835 pixels/metre vertical	
2Eh	4	00 00 00 00	0 colors	Number of colors in the palette
32h	4	00 00 00 00	0 important colors	0 means all colors are important
Start of pixel array (bitmap data)				
36h	3	00 00 FF	0 0 255	Red, Pixel (x=0, y=1)
39h	3	FF FF FF	255 255 255	White, Pixel (x=1, y=1)
3Ch	2	00 00	0 0	Padding for 4 byte alignment (could be a value other than zero)
3Eh	3	FF 00 00	255 0 0	Blue, Pixel (x=0, y=0)
41h	3	00 FF 00	0 255 0	Green, Pixel (x=1, y=0)
44h	2	00 00	0 0	Padding for 4 byte alignment (could be a value other than zero)

Dasar Teori

BITMAP (BMP) Image

Dalam proyek ini, gambar menggunakan format BMP karena tidak perlu ada langkah dekompresi dan data pixel dapat langsung diakses dengan mudah.





Desain Sistem

Terdapat lima komponen utama, yaitu:

- **CPU** : Mengontrol alur instruksi (IDLE, FETCH, DECODE, EXECUTE, dan COMPLETE). Terdapat RESET juga untuk mengembalikan semua nilai signal ke nilai awal
- **Decoder** : Menerjemahkan instruksi ke opcode (jenis kernel mana yang dipakai) dan address_flag (jenis unit mana yang akan dieksekusi)
- **ALU** : Melakukan operasi convolution secara direct
- **MMU** : Melakukan operasi convolution secara im2col
- **Line Buffer** : Menyimpan baris piksel menggunakan BRAM untuk membentuk window convolution, dan menangani padding pada tepi gambar.

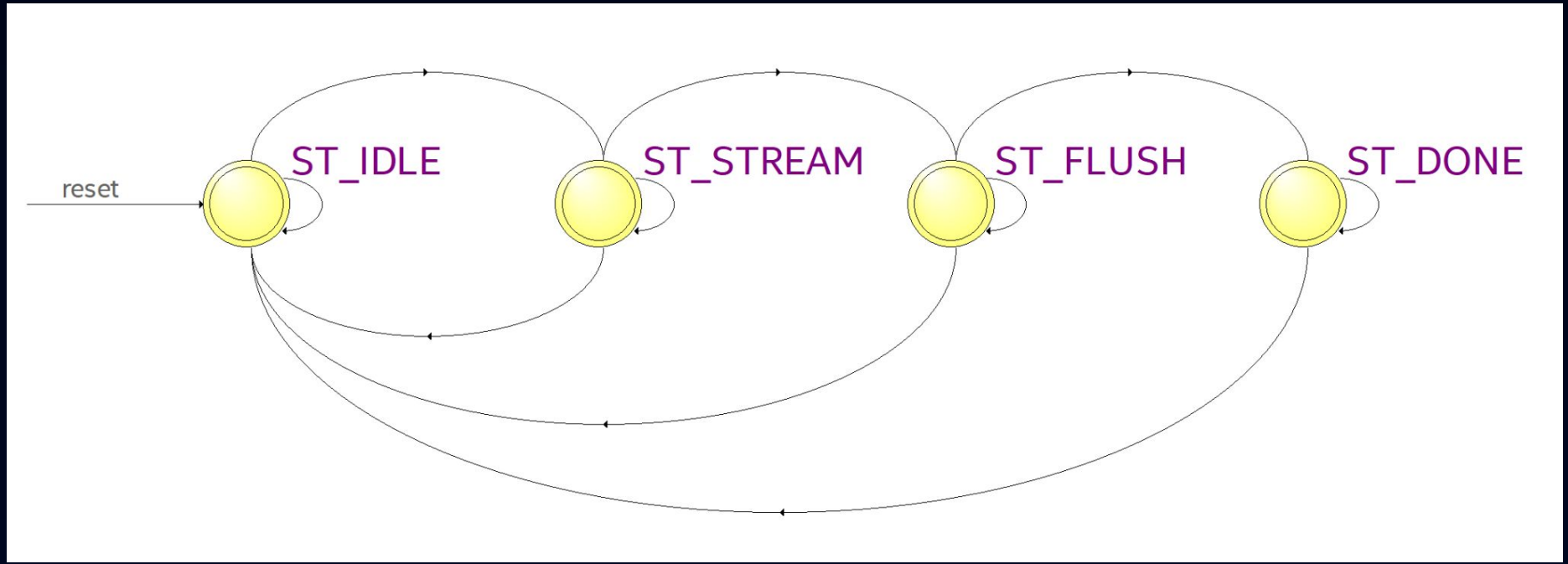


Mapping Instruction

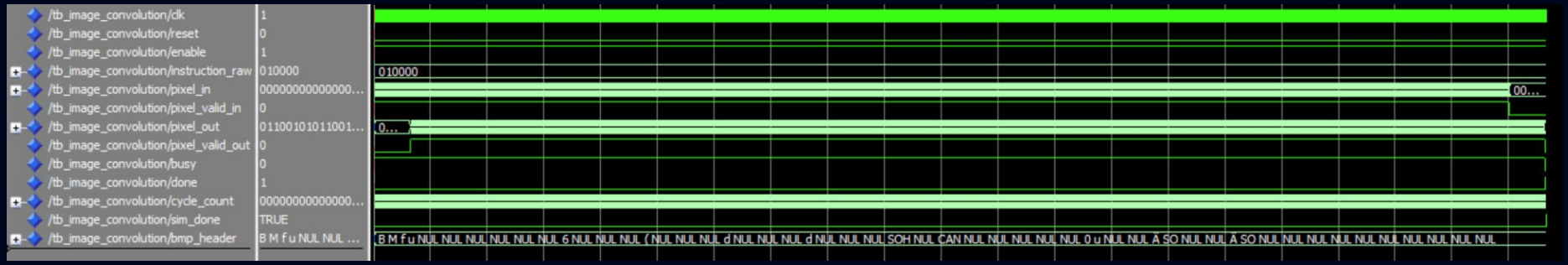
Size	Instruction (ALU)	Instruction (MMU)	Filter Name
1×1	`000000`	`001000`	Identity / Passthrough
3×3	`010000`	`011000`	Box Blur
3×3	`010001`	`011001`	Horizontal Edge
3×3	`010010`	`011010`	Vertical Edge
3×3	`010011`	`011011`	Laplacian
3×3	`010100`	`011100`	Sharpening
3×3	`010101`	`011101`	Sobel Horizontal
3×3	`010110`	`011110`	Sobel Vertical
3×3	`010111`	`011111`	Custom Kernel
5×5	`100000`	`101000`	Box Blur 5x5
5×5	`100001`	`101001`	Horizontal Edge 5x5
5×5	`100010`	`101010`	Vertical Edge 5x5
5×5	`100011`	`101011`	Laplacian 5x5
5×5	`100100`	`101100`	Sharpening 5x5
5×5	`100101`	`101101`	Sobel Horizontal 5x5
5×5	`100110`	`101110`	Sobel Vertical 5x5
5×5	`100111`	`101111`	Gaussian Blur 5x5
7×7	`110000`	`111000`	Box Blur 7x7
7×7	`110001`	`111001`	Horizontal Edge 7x7
7×7	`110010`	`111010`	Vertical Edge 7x7
7×7	`110011`	`111011`	Laplacian 7x7
7×7	`110100`	`111100`	Sharpening 7x7
7×7	`110101`	`111101`	Sobel Horizontal 7x7
7×7	`110110`	`111110`	Sobel Vertical 7x7
7×7	`110111`	`111111`	Custom/Gaussian 7x7

Hasil Pengujian

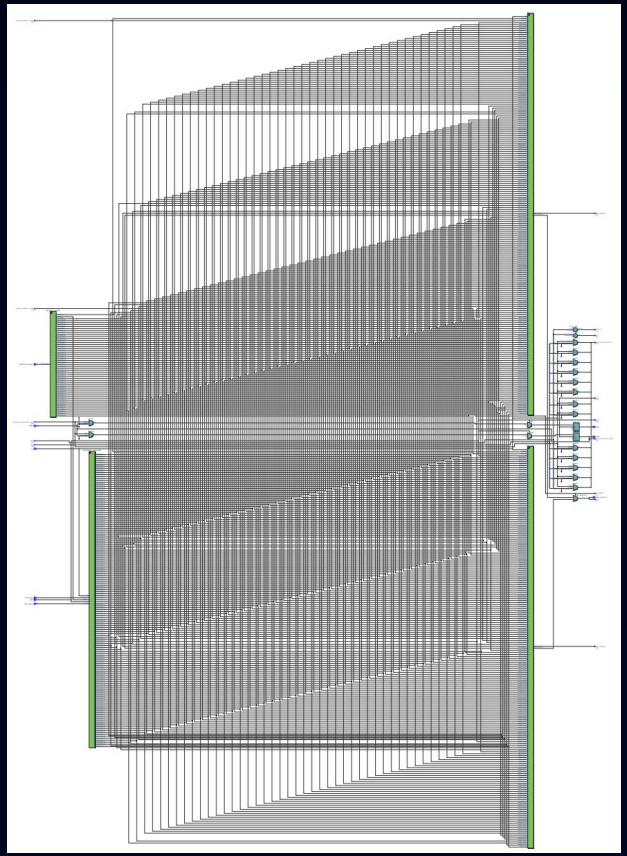
State Diagram



ModelSim Waveform



Synthesis



Box Blur

$$K = \begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix}$$

Kernel yang dipakai



Original



3x3



5x5



7x7

Horizontal Edge

$$K = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Kernel yang dipakai



Original



3x3



5x5



7x7

Vertical Edge

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Kernel yang dipakai



Original



3x3



5x5



7x7

Laplacian

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Kernel yang dipakai



Original



3x3



5x5



7x7

Sobel Horizontal

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Kernel yang dipakai



Original



3x3



5x5



7x7

Sobel Vertical

$$K = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Kernel yang dipakai



Original



3x3



5x5



7x7

Sharpening

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Kernel yang dipakai



Original



3x3

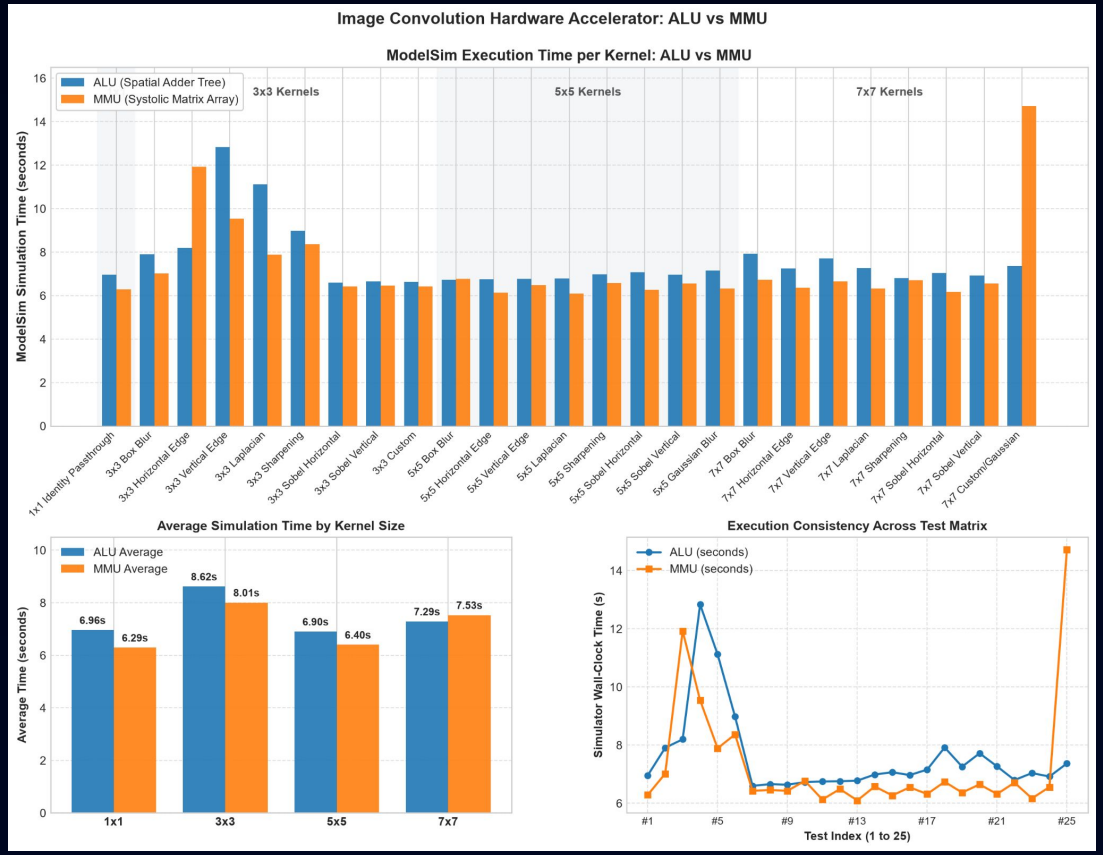


5x5



7x7

ALU vs MMU


Image Convolution Hardware Accelerator: ALU vs MMU
ModelSim Execution Time per Kernel: ALU vs MMU
ALU (Spatial Adder Tree)
MMU (Systolic Matrix Array)
3x3 Kernels
5x5 Kernels
7x7 Kernels
ModelSim Simulation Time (seconds)
1x1 Identity Passthrough
3x3 Box Blur
3x3 Horizontal Edge
3x3 Vertical Edge
3x3 Laplacian
3x3 Sharpening
3x3 Sobel Horizontal
3x3 Sobel Vertical
3x3 Custom
5x5 Box Blur
5x5 Horizontal Edge
5x5 Vertical Edge
5x5 Laplacian
5x5 Sharpening
5x5 Sobel Horizontal
5x5 Sobel Vertical
5x5 Gaussian Blur
7x7 Box Blur
7x7 Horizontal Edge
7x7 Vertical Edge
7x7 Laplacian
7x7 Sharpening
7x7 Sobel Horizontal
7x7 Sobel Vertical
7x7 Custom/Gaussian
Average Simulation Time by Kernel Size
ALU Average
MMU Average
Average Time (seconds)
6.96s
6.29s
8.62s
8.01s
6.90s
6.40s
7.29s
7.53s
1x1
3x3
5x5
7x7
Execution Consistency Across Test Matrix
ALU (seconds)
MMU (seconds)
Simulator Wall-Clock Time (s)
Test Index (1 to 25)

Thank You